

Enterprise HR & Payroll Management System - Advanced OOP Case Study

DETAILED PROBLEM EXPLANATION

You are required to design and implement an Enterprise HR & Payroll Management System for a mid-sized organization. The company employs Full-Time Employees, Contract Employees, and Managers. Each employee category follows different salary rules.

The system must validate employee data before storing it. Employee names must have minimum length, department must not be empty, and salary values must not be negative.

Salary calculation must follow polymorphism principles. Full-Time Employees receive Base Salary + HRA + Bonus minus Tax (based on slabs). Contract Employees are paid Hourly Rate multiplied by Hours Worked. Managers extend Full-Time Employees and receive additional Leadership Bonus.

Tax Slabs:

- Less than 50,000 -> 5%
- Between 50,000 and 100,000 -> 10%
- Above 100,000 -> 20%

The system must also provide analytics:

- Total Payroll Cost
- Top 3 Highest Paid Employees
- Average Salary by Department
- Payroll Report

Students must use:

- Abstraction (Employee base class)
- Inheritance (FullTimeEmployee, ContractEmployee, Manager)
- Polymorphism (calculateSalary method)
- Encapsulation (protected fields, validation logic)
- Comparable (sorting employees by salary)
- Stream API (analytics)
- Custom Exceptions (business rule validation)

All missing TODO sections must be implemented by students.

----- PayrollException.java -----

```
public class PayrollException extends Exception {
    public PayrollException(String message) {
        super(message);
    }
}

public class InvalidEmployeeException extends PayrollException {
    public InvalidEmployeeException(String message) {
        super(message);
    }
}

public class InvalidSalaryException extends PayrollException {
    public InvalidSalaryException(String message) {
        super(message);
    }
}
```

----- Employee.java -----

```
public abstract class Employee implements Comparable<Employee> {

    protected int id;
    protected String name;
    protected String department;
    protected double baseSalary;

    public Employee(int id, String name,
                    String department,
                    double baseSalary)
        throws InvalidEmployeeException {

        if (id <= 0)
            throw new InvalidEmployeeException("Invalid ID");

        // TODO:
        // Validate name length >= 3
        // Validate department not null or empty
        // Validate baseSalary >= 0

        this.id = id;
        this.name = name;
        this.department = department;
        this.baseSalary = baseSalary;
    }

    public abstract double calculateSalary()
        throws InvalidSalaryException;

    @Override
    public int compareTo(Employee other) {

        // TODO:
        // Sort in descending order of salary
        return 0;
    }

    public String getDepartment() {
        return department;
    }
}
```

----- FullTimeEmployee.java -----

```
public class FullTimeEmployee extends Employee {

    private double hra;
    private double bonus;

    public FullTimeEmployee(int id, String name,
                            String dept,
                            double baseSalary,
                            double hra,
                            double bonus)
```

```

        throws InvalidEmployeeException {

        super(id, name, dept, baseSalary);

        // TODO:
        // Validate hra >= 0
        // Validate bonus >= 0

        this.hra = hra;
        this.bonus = bonus;
    }

    @Override
    public double calculateSalary()
        throws InvalidSalaryException {

        // TODO:
        // gross = baseSalary + hra + bonus
        // tax = applyTax(gross)
        // return gross - tax

        return 0;
    }

    protected double applyTax(double gross) {

        // TODO:
        // Apply slab logic
        return 0;
    }
}

```

----- ContractEmployee.java -----

```

public class ContractEmployee extends Employee {

    private double hourlyRate;
    private int hoursWorked;

    public ContractEmployee(int id, String name,
                            String dept,
                            double hourlyRate,
                            int hoursWorked)
        throws InvalidEmployeeException {

        super(id, name, dept, 0);

        // TODO:
        // Validate hourlyRate > 0
        // Validate hoursWorked >= 0

        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    @Override
    public double calculateSalary()
        throws InvalidSalaryException {

        // TODO:
        // return hourlyRate * hoursWorked
        return 0;
    }
}

```

----- Manager.java -----

```

public class Manager extends FullTimeEmployee {

    private double leadershipBonus;

    public Manager(int id, String name,
                  String dept,
                  double baseSalary,
                  double hra,
                  double bonus,

```

```

        double leadershipBonus)
        throws InvalidEmployeeException {

    super(id, name, dept, baseSalary, hra, bonus);
    this.leadershipBonus = leadershipBonus;
}

@Override
public double calculateSalary()
    throws InvalidSalaryException {

    // TODO:
    // Get full-time salary
    // Add leadershipBonus
    return 0;
}
}

```

----- PayrollService.java -----

```

import java.util.*;
import java.util.stream.*;

public class PayrollService {

    private List<Employee> employees = new ArrayList<>();

    public void addEmployee(Employee emp) {
        employees.add(emp);
    }

    public double totalPayrollCost() {

        // TODO:
        // Use Stream API
        return 0;
    }

    public List<Employee> top3HighestPaid() {

        // TODO:
        // Sort and limit 3
        return null;
    }

    public Map<String, Double> averageSalaryByDepartment() {

        // TODO:
        // groupingBy department
        return null;
    }

    public void payrollReport() {

        // TODO:
        // Print total payroll
        // Print department averages
        // Print top 3 employees
    }
}

```

----- MainApplication.java -----

```

public class MainApplication {

    public static void main(String[] args) throws Exception {

        PayrollService service = new PayrollService();

        Employee e1 = new FullTimeEmployee(
            1, "Arjun", "IT",
            60000, 10000, 5000);

        Employee e2 = new ContractEmployee(
            2, "Ravi", "Finance",
            500, 120);
    }
}

```

```
Employee e3 = new Manager(
    3, "Meera", "IT",
    90000, 15000, 10000,
    20000);

service.addEmployee(e1);
service.addEmployee(e2);
service.addEmployee(e3);

service.payrollReport();
    }
}
```